Project 1 Performance Modeling and Analysis

1. Introduction

Cloud GPUs are often compared by their published peak numbers. This project tests how well those numbers predict training performance. We train three depths of one network family, ResNet-50, ResNet-101, and ResNet-152, on two GPUs with different hardware balance: the NVIDIA L4 has the higher published FP32 peak (30.3 vs. 19.5 TFLOP/s), while the A100 40 GB has 5.2 times its memory bandwidth (1,555 vs. 300 GB/s).

We ask three questions. How do training time, throughput, and memory use change with GPU and model depth? Which hardware limit, computation or memory bandwidth, explains each result? How do the operations the GPU actually executes compare with an analytical count of the model's work? We answer them with timed training steps, Nsight Compute profiles of the forward and backward passes, and Roofline analysis at both the model and the kernel level.

2. Background and Related Work

2.1 Related Work

Convolutional neural network (CNN) families have different design goals. VGG studies deeper networks built from small convolutional filters [7]. ResNet uses shortcut connections to make deeper networks easier to train [6]. MobileNetV2 is designed for mobile devices [8], EfficientNet scales depth, width, and image resolution together [9], and ConvNeXt updates the CNN design [10]. These differences help explain why models perform differently across hardware.

Earlier studies compare both hardware performance and how well models complete a task. Bianco et al. [1] compare image-recognition models using operation counts, parameter size, memory use, accuracy, and prediction time on two hardware platforms. Their study focuses on inference, meaning the use of a trained model to make predictions, and provides a useful basis for comparing models and hardware.

MLPerf Training [2] sets rules for the models, training tasks, timing, and target quality used in a benchmark. These rules show why hardware comparisons need a clearly defined task and measurement procedure. Coleman et al. [3] analyze DAWNBench, which measures the time needed to reach a target accuracy. They also use Roofline analysis to examine GPU use and limits caused by memory transfers and communication. These studies help connect measured performance with where execution time is spent.

Micikevicius et al. [4] show how mixed-precision training uses lower precision for some calculations and higher precision for others, maintaining model accuracy while using less memory.

2.2 Roofline Model

The Roofline model estimates the performance limit of a workload. It helps us understand whether faster calculations or faster access to GPU memory would be more useful [5].

Arithmetic intensity, also called operational intensity, is the number of calculations performed for each byte transferred to or from GPU memory.

The formula combines these two limits [5]:
Attainable GFlops/sec = Min(Peak Floating Point Performance, Peak Memory Bandwidth × Operational Intensity)
The workload cannot run faster than either limit, so the smaller value sets its performance ceiling. We compare our measurements with this ceiling to see how much of the GPU's capacity the workload uses.

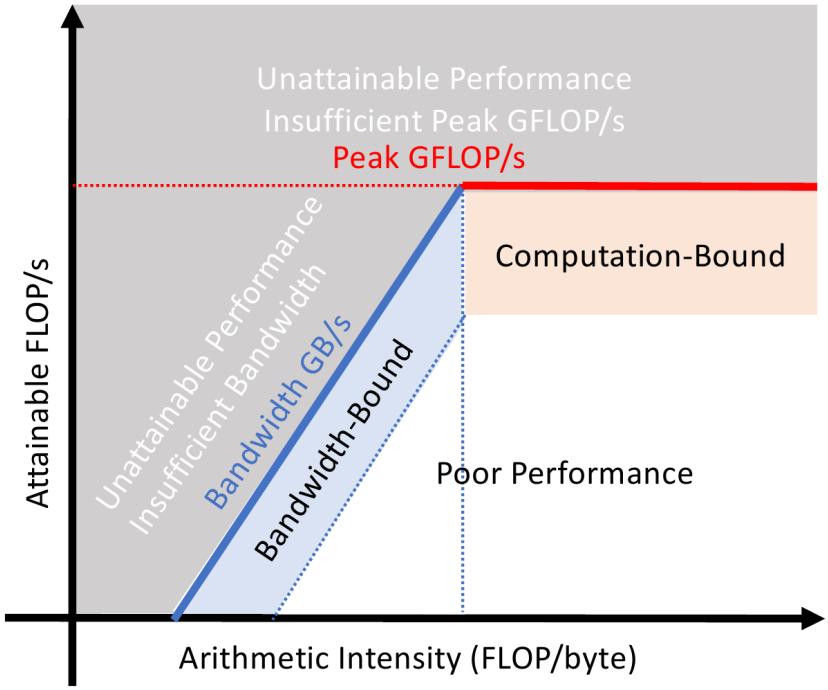


Figure 1. Roofline model.

In Figure 1, the sloped line shows the memory limit and the flat line shows the computing limit. They meet at the ridge point, where the two limits are equal [5].

2.3 ResNet Models

ResNet offers models with different numbers of layers within one family [6]. We use these models to study how adding layers changes the amount of computation and GPU performance.

ResNet was designed to make deep networks easier to train. Its shortcut connections add a block's input to the output of its layers, an approach called residual learning. He et al. [6] write this as $y = F(x, \{W_i\}) + x$, where x and y are the input and output, and F represents the block's calculation using weights $\{W_i\}$. Their experiments show that these connections make deeper networks easier to train [6].

ResNet models differ in their numbers of layers and the blocks they use. ResNet-18 and ResNet-34 use basic blocks with two main convolutional layers. ResNet-50, ResNet-101, and ResNet-152 use bottleneck blocks with three main convolutional layers. The model names give the conventional count of layers with learned weights [6].

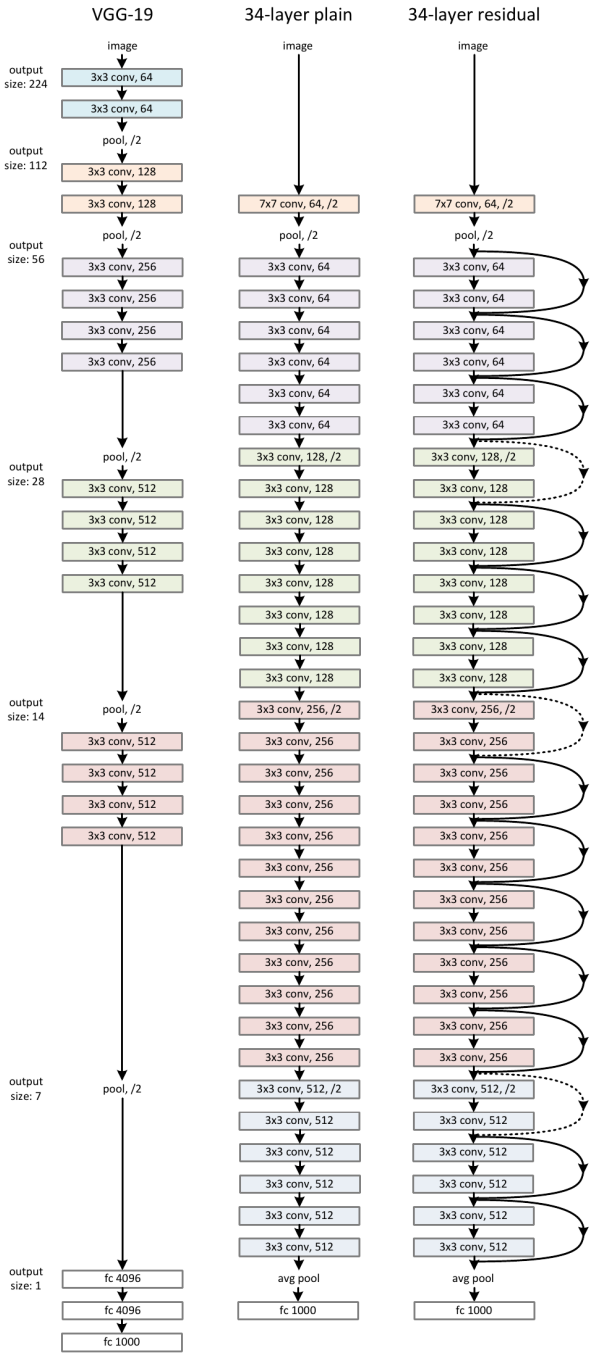


Figure 2. ImageNet network architectures.

3. Methodology

Component	Configuration	Purpose
Environment	NVIDIA L4 and A100 40 GB	Compare how different GPUs perform on the same workload.
Model	ResNet-50, ResNet-101, and ResNet-152	Study how network depth affects performance within one model family.
Batch size	32	Keep the number of images per training step fixed across comparisons.
Data	A small, fixed ImageNet subset	Use the same images and preprocessing across all experiments.
Precision	FP32 (32-bit floating point)	Use the same numerical precision in every experiment.

3.1 Experimental Design and Setup

We compare ResNet-50, ResNet-101, and ResNet-152 on NVIDIA L4 and A100 40 GB GPUs. These six combinations show how training performance changes with GPU type and the number of model layers.

1. Load the three ResNet models in PyTorch and Torchvision and prepare the ImageNet subset.

2. Estimate the forward calculations for each model using the method in Section 3.2.

3. After warm-up, measure training-step time, images processed per second, and peak GPU memory use for each model on each GPU.

4. Use NVIDIA Nsight Compute in separate runs to measure arithmetic operations, GPU memory transfers, and the time of individual GPU programs (kernels) during the forward and backward passes.

5. Draw forward and backward Roofline plots for each GPU and compare the results across GPUs and model depths.

3.2 Workload Complexity

The image below reproduces the original ResNet architecture table for all five models [6]. It shows layer sizes, convolutional blocks, and the number of times each block is repeated for a 224 × 224 RGB image.

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Table 1. ResNet architectures [6, Table 1].

We use Torchvision's ResNet V1.5 with batch size 32 and FP32, training all layers. It reduces image dimensions in a different layer from the original ResNet, which changes the operation count. The table above shows the paper's published counts; our estimates follow V1.5.

Forward FLOPs per layer

$$\mathrm{FLOPs_{conv}} = 2K_h K_w C_{in} C_{out} H_{out} W_{out}$$

$\mathrm{FLOPs_{conv}}$ counts forward operations for one convolution and one image; K_h and K_w are kernel height and width; C_{in} and C_{out} are input and output channels; H_{out} and W_{out} are output height and width. Each output value requires $K_h \times K_w \times C_{in}$ multiply-adds. Multiplying by the number of output values and counting two FLOPs per multiply-add gives the layer total. These convolutions have one group.

$$\mathrm{FLOPs_{fc}} = 2D_{in} D_{out}$$

$\mathrm{FLOPs_{fc}}$ counts the classifier's forward operations for one image; D_{in} and D_{out} are its input and output sizes (2,048 and 1,000). Each output combines all inputs, giving $D_{in} \times D_{out}$ multiply-adds.

Total forward FLOPs per image

$$\mathrm{FLOPs} = \sum_j r_j \mathrm{FLOPs}_j$$

FLOPs is the total forward operation count per image; j identifies a layer or group of blocks; r_j is its repetition count; FLOPs_j counts forward operations for one layer or block in that group. Add the convolutions in each block, including any projection shortcut. Multiply by the number of repetitions, then add the group totals to get the model total.

Batch forward FLOPs

$$\mathrm{FLOPs}_{\mathrm{batch}} = \mathrm{B} \times \mathrm{FLOPs}$$

$\mathrm{FLOPs}_{\mathrm{batch}}$ counts forward operations per batch; B is batch size (32); FLOPs counts forward operations per image. Multiply the per-image count by B to get the batch total.

Each row shows the input size, output size, and calculation count for one layer or group of repeated blocks. C, H, and W mean channels, height, and width. The calculation count includes all repetitions in that row and any shortcut convolution.

ResNet-50 calculations

Layer / block	Count	Input			Output			Forward GFLOPs per image
		C	H	W	C	H	W	
conv1	1	3	224	224	64	112	112	0.236
Max pool	1	64	112	112	64	56	56	Not counted
conv2_1	1	64	56	56	256	56	56	0.462
conv2_2–3	2	256	56	56	256	56	56	0.873
conv3_1	1	256	56	56	512	28	28	0.745
conv3_2–4	3	512	28	28	512	28	28	1.310
conv4_1	1	512	28	28	1,024	14	14	0.745
conv4_2–6	5	1,024	14	14	1,024	14	14	2.184
conv5_1	1	1,024	14	14	2,048	7	7	0.745
conv5_2–3	2	2,048	7	7	2,048	7	7	0.873
Average pool	1	2,048	7	7	2,048	1	1	Not counted
Classifier	1	2,048	1	1	1,000	1	1	0.004
Model total	—	—	—	—	—	—	—	**8.178**

ResNet-101 calculations

Layer / block	Count	Input			Output			Forward GFLOPs per image
		C	H	W	C	H	W	

Layer / block	Count	Input			Output			Forward GFLOPs per image
		C	H	W	C	H	W	
conv1	1	3	224	224	64	112	112	0.236
Max pool	1	64	112	112	64	56	56	Not counted
conv2_1	1	64	56	56	256	56	56	0.462
conv2_2–3	2	256	56	56	256	56	56	0.873
conv3_1	1	256	56	56	512	28	28	0.745
conv3_2–4	3	512	28	28	512	28	28	1.310
conv4_1	1	512	28	28	1,024	14	14	0.745
conv4_2–23	22	1,024	14	14	1,024	14	14	9.608
conv5_1	1	1,024	14	14	2,048	7	7	0.745
conv5_2–3	2	2,048	7	7	2,048	7	7	0.873
Average pool	1	2,048	7	7	2,048	1	1	Not counted
Classifier	1	2,048	1	1	1,000	1	1	0.004
Model total	**—**	**—**	**—**	**—**	**—**	**—**	**—**	**15.603**

ResNet-152 calculations

Layer / block	Count	Input			Output			Forward GFLOPs per image
		C	H	W	C	H	W	
conv1	1	3	224	224	64	112	112	0.236
Max pool	1	64	112	112	64	56	56	Not counted
conv2_1	1	64	56	56	256	56	56	0.462
conv2_2–3	2	256	56	56	256	56	56	0.873
conv3_1	1	256	56	56	512	28	28	0.745
conv3_2–8	7	512	28	28	512	28	28	3.057
conv4_1	1	512	28	28	1,024	14	14	0.745
conv4_2–36	35	1,024	14	14	1,024	14	14	15.286
conv5_1	1	1,024	14	14	2,048	7	7	0.745
conv5_2–3	2	2,048	7	7	2,048	7	7	0.873
Average pool	1	2,048	7	7	2,048	1	1	Not counted
Classifier	1	2,048	1	1	1,000	1	1	0.004
Model total	**—**	**—**	**—**	**—**	**—**	**—**	**—**	**23.027**

Batch FLOP estimates

Model	Forward GFLOPs per batch of 32
ResNet-50	261.708
ResNet-101	499.290
ResNet-152	736.872

The estimates cover forward-pass calculations in the convolutional layers and final classifier. The Roofline analysis uses measured operations from both the forward and backward passes.

3.3 Measurement Procedure

The forward pass calculates model outputs, and the backward pass calculates gradients. Updating the model's weights is timed as part of the full training step but is excluded from the Roofline plots.

1. Run warm-up steps for each model on each GPU so initialization and kernel selection finish before timing.

2. Use CUDA events to time gradient reset, the forward pass, loss calculation, the backward pass, and the optimizer update. Wait for GPU work to finish before reading the time.

3. Restore the model and optimizer to the same starting states before each repetition. Report step time, images per second, and peak allocated GPU memory for each repetition.

4. After warm-up, use NVIDIA Nsight Compute to measure the forward and backward passes separately. Collect operations, memory transfers, and execution time for the same kernel calls.

For the measures below, n is the number of timed steps and T is their total time in seconds, measured with CUDA events; batch size is 32. Save profiling results separately for each model, GPU, and pass.

Metric	Unit	Calculation or measurement
Mean step time	ms / step	1,000 × T/n, using CUDA-event intervals.
Image throughput	images / s	32n/T, using the same timed GPU interval.
GPU speedup	ratio	Average L4 step time / average A100 step time for the same model.
Peak allocated memory	MiB	After warm-up and before timing, reset the peak-memory counter. Read max_memory_allocated() and convert to MiB.
Measured operations	FLOPs	Nsight Compute counts of floating-point additions and multiplications.
GPU memory transfers	bytes	Nsight Compute DRAM bytes read plus bytes written.
Kernel duration	µs	Nsight Compute time for the same kernel call used to count operations and memory transfers.

3.4 Roofline Construction

1. Draw a roof for each GPU using its peak FP32 calculation rate and memory bandwidth. The sloped line is bandwidth × arithmetic intensity; the flat line is the peak compute rate [5].

2. For each model and pass, add the measured FLOPs, GPU memory bytes read and written, and execution times across its kernels. Use the same kernel calls for all three totals.

3. Divide total FLOPs by total transferred bytes to get arithmetic intensity (FLOP/byte). Divide total FLOPs by the summed kernel time in seconds to get the measured calculation rate (FLOP/s) [11].

4. Plot the forward and backward results in separate panels for each GPU. Use the same roof and axis scales, show rates in GFLOP/s, and label each ResNet model [11].

5. To see what each combined point contains, group the same kernels by name into convolution and matrix-multiplication kernels (including cuDNN helper kernels) and batch-normalization and element-wise kernels (ReLU, residual addition, and their gradients). Compute the same totals for each group, and plot every kernel on one Roofline per GPU (Figure 5).

4. Results and Analysis

These results compare training time, image throughput, and memory use. The Roofline plots relate measured calculation rates to GPU memory transfers.

4.1 Training Performance and Memory Use

Table 2. Training measurements (step times averaged over 50 steps).

Model	GPU	Repeat	Average step time (ms)	Throughput (images/s)	Peak allocated memory (MiB)
ResNet-50	L4	1	167.991	190.49	2,938.93
ResNet-50	L4	2	168.677	189.71	2,940.75
ResNet-50	L4	3	169.598	188.68	2,940.75
ResNet-50	A100	1	71.974	444.60	2,967.56
ResNet-50	A100	2	72.000	444.45	2,970.68
ResNet-50	A100	3	71.957	444.71	2,970.68
ResNet-101	L4	1	267.856	119.47	4,332.19
ResNet-101	L4	2	269.278	118.84	4,331.69
ResNet-101	L4	3	271.504	117.86	4,331.69
ResNet-101	A100	1	118.268	270.57	4,354.69
ResNet-101	A100	2	118.272	270.56	4,358.94
ResNet-101	A100	3	118.305	270.49	4,358.94
ResNet-152	L4	1	388.877	82.29	6,001.67
ResNet-152	L4	2	394.398	81.14	6,005.42
ResNet-152	L4	3	392.327	81.56	6,005.42
ResNet-152	A100	1	165.758	193.05	6,021.05
ResNet-152	A100	2	165.722	193.09	6,024.92
ResNet-152	A100	3	165.717	193.10	6,024.92

A100 completes training steps faster than L4 for all three models, while deeper models take more time and use more memory on both GPUs.

4.2 Forward and Backward Roofline Analysis

Table 3 lists the hardware limits used to draw each GPU's Roofline.

Table 3. GPU specifications.

GPU	Peak FP32 (GFLOP/s)	DRAM bandwidth (GB/s)	Ridge point (FLOP/byte)
L4 [12]	30,300	300	101.00
A100 SXM4 40 GB [13]	19,500	1,555	12.54

Each point combines the measurements from one model's forward or backward kernels.

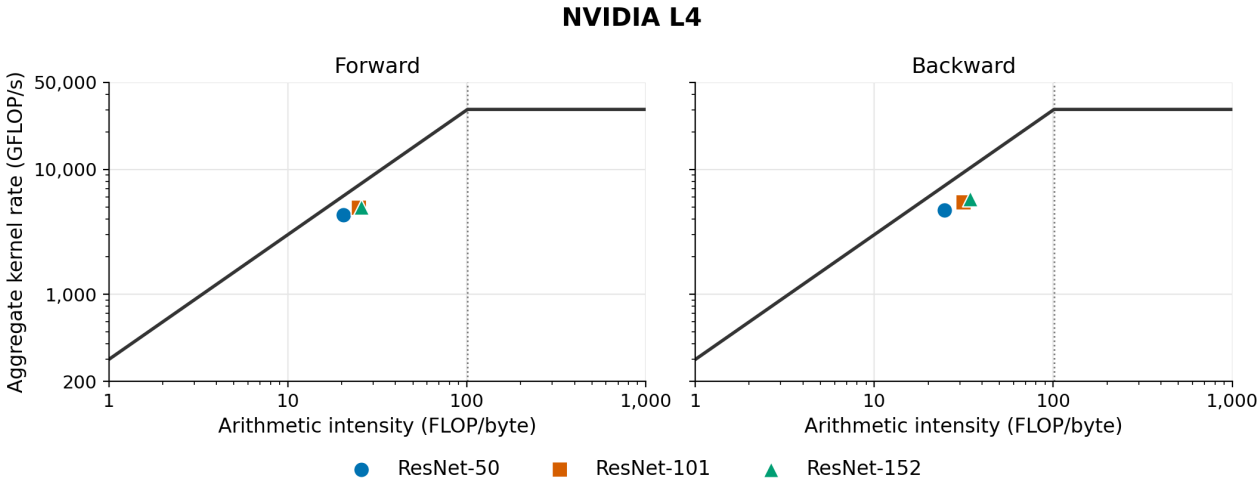


Figure 3. L4 Roofline analysis.

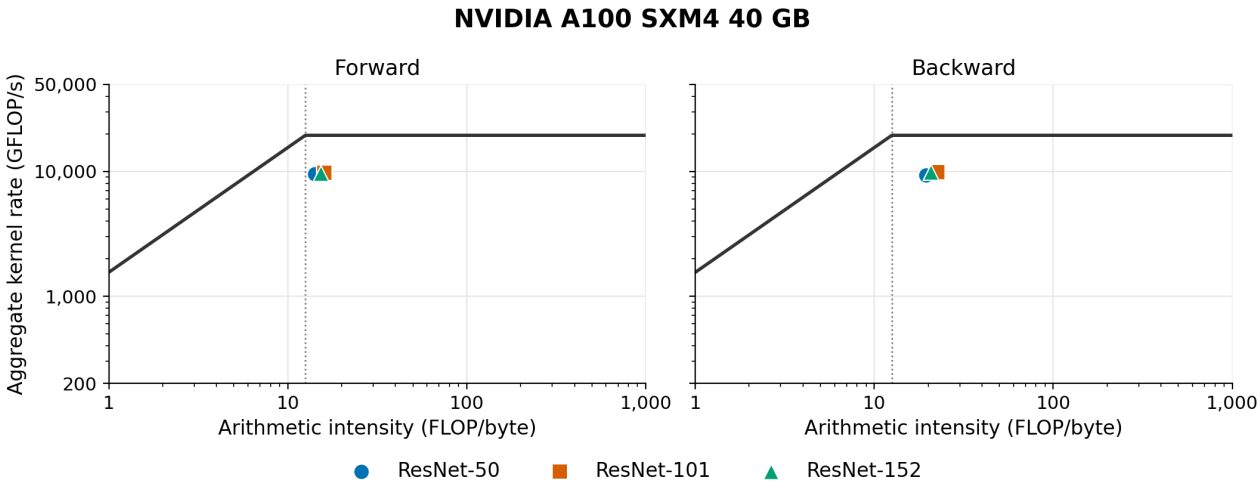


Figure 4. A100 Roofline analysis.

Table 4. Roofline results.

GPU	Model	Phase	Intensity (FLOP/byte)	Rate (GFLOP/s)	Roof fraction*
L4	ResNet-50	Forward	20.47	4,346	70.8%
L4	ResNet-50	Backward	24.62	4,742	64.2%
L4	ResNet-101	Forward	25.02	4,955	66.0%
L4	ResNet-101	Backward	31.64	5,438	57.3%
L4	ResNet-152	Forward	25.72	5,009	64.9%
L4	ResNet-152	Backward	34.35	5,815	56.4%
A100	ResNet-50	Forward	14.14	9,520	48.8%
A100	ResNet-50	Backward	19.40	9,377	48.1%
A100	ResNet-101	Forward	15.98	9,800	50.3%
A100	ResNet-101	Backward	22.57	9,865	50.6%
A100	ResNet-152	Forward	15.30	9,676	49.6%
A100	ResNet-152	Backward	20.66	9,892	50.7%

* Roof fraction is the measured calculation rate divided by the Roofline limit at that intensity.

The chart suggests that the workload is memory-bandwidth-bound on L4. Deeper models perform more calculations per byte and per second, but reach a smaller share of the limit set by memory bandwidth. ResNet-50 comes closest to that limit in both passes. To improve performance, one could consider reducing data transfer between GPU memory and the compute units.

On A100, the points sit near the ridge. This suggests a closer match between the workload and the GPU's balance of memory bandwidth and computing capacity. The points are close together, so changing model depth makes little difference to the achieved calculation rate.

Each point, however, combines hundreds of kernels with different behavior, so its position does not show which limit applies to the kernels that take most of the time. Section 4.3 separates them.

4.3 Kernel-Level Analysis

Table 5 and Figure 5 split the profiled kernels into the two groups defined in Section 3.4.

Table 5. Kernel time by kernel group (forward and backward passes combined).

GPU	Model	Conv/GEMM share of time	Conv/GEMM rate, GFLOP/s (% of peak)	BN + element-wise share of time	BN + element-wise DRAM GB/s (% of peak)
L4	ResNet-50	56.1%	8,115 (27%)	42.8%	277 (92%)
L4	ResNet-101	62.1%	8,431 (28%)	37.2%	278 (93%)
L4	ResNet-152	63.1%	8,737 (29%)	36.4%	277 (92%)
A100	ResNet-50	71.7%	12,959 (66%)	27.2%	1,105 (71%)
A100	ResNet-101	75.8%	12,847 (66%)	23.5%	1,114 (72%)

GPU	Model	Conv/GEMM share of time	Conv/GEMM rate, GFLOP/s (% of peak)	BN + element-wise share of time	BN + element-wise DRAM GB/s (% of peak)
A100	ResNet-152	75.5%	12,871 (66%)	24.0%	1,133 (73%)

The two groups hit different limits. Batch normalization and element-wise kernels do little arithmetic but take a large share of time, and on L4 they run close to peak memory bandwidth. L4 convolutions, by contrast, reach neither roof: even the fastest L4 kernel stays below half of the published FP32 peak, while A100 convolutions come close to theirs (Figure 5). L4's published peak therefore overstates what these FP32 kernels achieve.

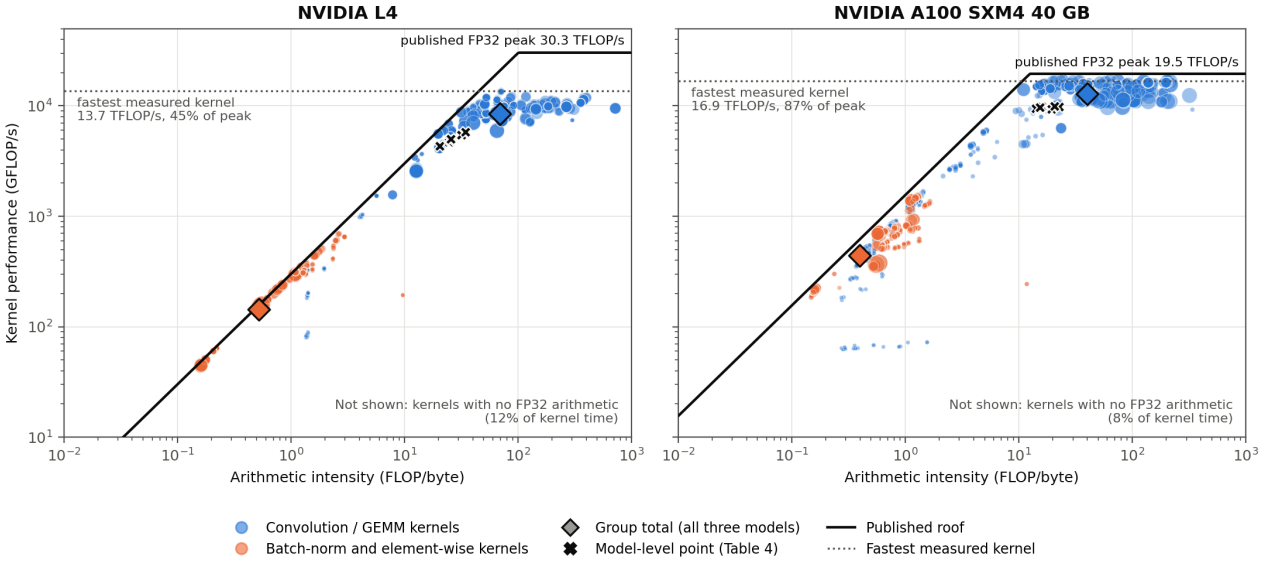


Figure 5. Kernel-level Roofline for all three models, forward and backward passes.

Table 6. A100 speedup over L4 by kernel group.

Model	Training step	All profiled kernels	Conv/GEMM kernels	BN + element-wise kernels	BN + element-wise share of L4's extra kernel time
ResNet-50	2.34×	2.21×	1.73×	3.48×	56%
ResNet-101	2.28×	2.07×	1.70×	3.28×	50%
ResNet-152	2.36×	2.06×	1.72×	3.13×	48%

Training-step speedup uses the mean step times in Table 2. Kernel speedups use summed profiled kernel time.

A100's lead is about twice as large on batch normalization and element-wise kernels as on convolutions (Table 6), and these memory-bound kernels account for about half of L4's extra time. Reducing their memory traffic, through kernel fusion or 16-bit data, would help L4 most.

4.4 Model Complexity Discussions

Table 7 brings our estimates and measurements together.

Table 7. Forward operation counts.

Model	Estimated GFLOPs	L4 measured GFLOPs	A100 measured GFLOPs
ResNet-50	261.708	212.925	208.825

Model	Estimated GFLOPs	L4 measured GFLOPs	A100 measured GFLOPs
ResNet-101	499.290	376.904	364.687
ResNet-152	736.872	535.064	519.846

The measured counts are lower than our estimates because the profiler counts the arithmetic the GPU actually performs. The GPU's math library often computes convolutions with shortcut methods that reach the same result with fewer operations, and it chooses these methods separately on each GPU. The two GPUs therefore do slightly different amounts of arithmetic for the same training step, so images processed per second is the fairest way to compare them.

A100 performed better than L4 throughout this analysis. Roofline results, however, are specific to both the hardware and the application, so a different application could show the opposite result.

Adding layers does not substantially improve the achieved calculation rate on A100, although step time grows more slowly than the analytical operation count. For a task where training-step speed and memory use are the priorities, ResNet-50 is still favored over the deeper models in our comparison.

5. Conclusion and Future Work

A100 trained all three ResNet models more than twice as fast as L4, even though L4 has the higher compute roof (published FP32 peak) in the Roofline model. The model-level Roofline explains part of this: on L4, every model falls on the memory-bandwidth side of the roof, where A100's much higher bandwidth gives it the advantage. The kernel-level analysis then shows more detail. Batch normalization and element-wise kernels are limited by memory bandwidth and cause about half of L4's extra time, while L4's convolutions stay far below its compute roof, so its published peak overstates what it achieves on this workload.

Limitations. Because of GPU availability, L4 ran on a personal Google Cloud VM and A100 on NYU HPC through Open OnDemand, so host CPU, driver, and profiler versions also differed.

Future work could train with 16-bit numbers (mixed precision), which cuts memory traffic and is how these GPUs are usually used, and test a wider variety of models to understand which types of workloads suit each GPU.

6. References

[1] Bianco et al. *Benchmark Analysis of Representative Deep Neural Network Architectures*. IEEE Access, 2018.

[2] Mattson et al. *MLPerf Training Benchmark*. MLSys, 2020.

[3] Coleman et al. *Analysis of DAWNBench, a Time-to-Accuracy Machine Learning Performance Benchmark*. ACM SIGOPS Operating Systems Review, 2019.

[4] Micikevicius et al. *Mixed Precision Training*. ICLR, 2018.

[5] Williams et al. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. Communications of the ACM, 2009.

[6] He et al. *Deep Residual Learning for Image Recognition*. CVPR, 2016.

[7] Simonyan and Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition*. ICLR, 2015.

[8] Sandler et al. *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. CVPR, 2018.

[9] Tan and Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. ICML, 2019.

[10] Liu et al. *A ConvNet for the 2020s*. CVPR, 2022.

[11] Yang et al. *Hierarchical Roofline Performance Analysis for Deep Learning Applications*. Intelligent Computing, 2021.

[12] NVIDIA. NVIDIA L4 Tensor Core GPU. Product specifications. https://www.nvidia.com/en-us/data-center/l4/.

[13] Krashinsky et al. NVIDIA Ampere Architecture In-Depth. NVIDIA Technical Blog, May 14, 2020. https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/.

AI assistance: I used OpenAI Codex (Astra 6) to help me understand the requirements. I wrote an initial plan, and Codex helped refine it and assisted with the implementation. I used Claude (Opus 5.5) to assist with debugging and provide critiques. I reviewed the training results and drafted and revised the report, with Codex assisting with selected sections and tables. I made the final design decisions and take responsibility for the final analysis.

Project files: GitHub repository